



FUNDAMENTOS Y TEORÍA DE LA COMPUTACIÓN

Eficiencia de Algoritmos

- Un **algoritmo** es un conjunto de pasos (instrucciones) para resolver un problema.
 - Debe ser robusta y correcta!!!.
 - Debe ser eficiente!!!.
- El análisis de algoritmos nos va a permitir estimar cómo de eficiente es un algoritmo.
- Un problema puede tener diferentes soluciones (algoritmos)
- El análisis de algoritmos nos va a permitir **comparar algoritmos y elegir el más eficiente.**

Eficiencia de Algoritmos

- ¿Cómo estudiar el rendimiento de un algoritmo?:
 - **Complejidad temporal:** estima el tiempo requerido por un algoritmo.
 - **Complejidad espacial:** estima el espacio en memoria principal que necesita utilizar el algoritmo.
 - Es posible estudiar el coste de otros recursos: internet, impresora, etc.

Eficiencia de Algoritmos - Análisis empírico

- En el análisis empírico, se calcula cuánto tiempo tarda un algoritmo en resolver un problema para distintos tamaños de entrada. Pasos:
 1. Implementar el algoritmo
 2. Incluir instrucciones que permitan medir el tiempo de ejecución.
 3. Ejecutar el programa con entradas de diferentes tamaño y obtener los tiempos de ejecución para cada tamaño
 4. Importa los resultados a un fichero excel y mostrarlos en un gráfico (X Y dispersión)

Eficiencia de Algoritmos

- Dado un número n , desarrolla una función que sume los n primeros números:

1) Implementa el algoritmo:

```
def sum_n(n: int) -> int:
    """returns the sum of the first n numbers"""

    if not isinstance(n, int) or n < 0:
        return None

    result = 0
    for i in range(n+1):
        result += i
    return result
```

2) Incluye instrucciones para medir el tiempo:

```
import time

def sum_n(n: int) -> (int, float):
    start = time.time() # returns the time in milliseconds

    if not isinstance(n, int) or n < 0:
        return None

    result = 0

    for i in range(n+1):
        result += i
    end = time.time()
    return result, end-start # we also return the time
```

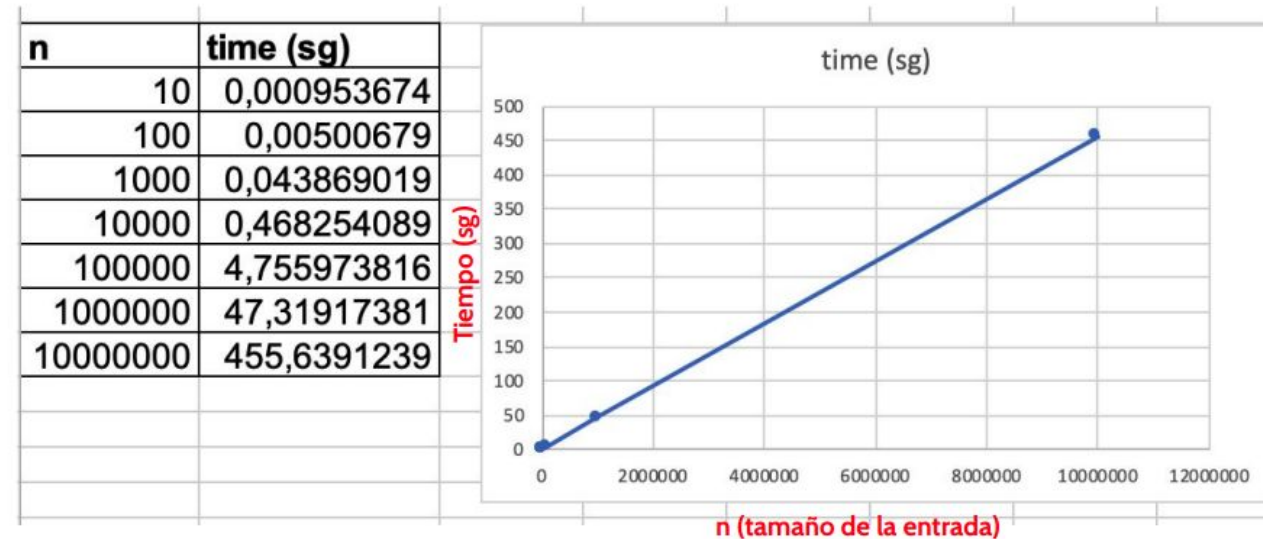
Eficiencia de Algoritmos

3) Ejecuta el programa para tamaños diferentes (mostramos el tiempo en segundos)

```
if __name__ == '__main__':  
    for i in range(1, 8):  
        input_size = pow(10, i)  
        result_sum, execution_time = sum_n(input_size)  
        print("sumN({})={}, time={} sg".format(input_size, result_sum, execution_time*100))
```

sumN(10)=55, time=0.00095367431640625 sg
sumN(100)=5050, time=0.0040531158447265625 sg
sumN(1000)=500500, time=0.04291534423828125 sg
sumN(10000)=50005000, time=0.45299530029296875 sg
sumN(100000)=5000050000, time=4.713773727416992 sg
sumN(1000000)=500000500000, time=49.21984672546387 sg
sumN(10000000)=50000005000000, time=456.09593391418457 sg

4) Importa los resultados a un fichero excel para mostrarlos en un gráfico



Eficiencia de Algoritmos - Análisis Teórico

- Usa pseudocódigo (no se implementan los algoritmos).
- Consiste en obtener la **función $T(n)$** , que representa el **número de operaciones** que deben ser ejecutadas para una **entrada de tamaño n** .
- Al ser una función, es posible considerar todas las posibles entradas (tamaños).
- Como no se miden tiempos de ejecución, es independiente del entorno Hardware/Software.

Eficiencia de Algoritmos

- ¿Cómo contamos operaciones / instrucciones en un algoritmo?.
- En cualquier algoritmo, nos podemos encontrar con los siguientes tipos de operaciones:
 - Operaciones primitivas
 - Operaciones bucles
 - Operaciones selectivas o condicionales (if)

Eficiencia de Algoritmos

- Ejemplos:

- Asignar valor a una variable: $x=2$
- Indexar un elemento en un array: $\text{vector}[3]$
- Devolver un valor en una función: $\text{return } x$
- Evaluar una expresión aritmética: $x+3$
- Evaluar una expresión lógica: $0 < i < \text{index}$

Este tipo de operaciones se van a contabilizar como una operación.

Eficiencia de Algoritmos

- ¿Qué ocurre si una operación primitiva se puede descomponer de varias operaciones primitivas?
return vector[3] + vector[5]
- En este caso, podríamos contar:
 - 1 operación por acceder a vector[3]
 - 1 operación por acceder a vector[5]
 - 1 operación por la operación de suma
 - 1 operación por el return.
- Es decir, en total tendríamos cuatro operaciones.

Eficiencia de Algoritmos

Operación	# operaciones primitivas
<code>x = 2</code>	1
<code>x = y + 3</code>	2 (1 suma + 1 asignación)
<code>return a[0] + 1</code>	3 (1 acceso + 1 suma + 1 return)
<code>return node is None or node.elem > 5</code>	5 (1 node is None + 1 obtener node.elem + 1 node.elem > 5 + 1 or + 1 return)
<code>print("Hola")</code>	1

Eficiencia de Algoritmos

- Si nuestro algoritmo se compone de varios bloques, su función $T(n)$, será la suma de las funciones $T(n)$ de cada bloque.

B1
B2
..
Bn

$$T(n) = T(B1) + T(B2) + \dots + T(Bn)$$

Algorithm *swap*(a, b)

operations

temp=a

1

a=b

1

b=temp

1

$$T(n) = 1 + 1 + 1 = 3$$

Eficiencia de Algoritmos

- La función $T(n)$ para un bucle se calculará como el número de veces que se ejecuta el bucle (número de iteraciones) por la función $T(n)$ del bloque interno B .

while condition:
 B

for x in ... :
 B

$$T_{\text{loop}}(n) = T(B) * \text{número de iteraciones}$$

- Por tanto, la función $T(n) = (n+1) * 2$.
- En este caso, la función temporal **sí depende del valor de n .**

```
for i in range(1, n+1):  
    result = result + i
```

Eficiencia de Algoritmos

- Dado el siguiente bucle anidado:

```
for i in range(n):  
    for i in range(n):  
        print(i*j)    #T(n) = 2
```

- Su función temporal $T(n)$ es:
$$T(n) = n * n * 2 = 2n^2$$
- Es decir, 2 del bucle interno, por el número de iteraciones del bucle interno, por el número de iteraciones del bucle externo.

Eficiencia de Algoritmos

- ¿Qué pasa si cada bucle se ejecuta un número distinto de veces?

```
for i in range(n1):  
    for i in range(n2):  
        print(i*j)
```

- En este caso, vamos a definir $n = \max(n_1, n_2)$,
- Podemos hacer la siguiente simplificación:

$$T(n) = n_1 * n_2 * 2 < n * n * 2 = 2n^2$$

Eficiencia de Algoritmos

- **If-Else:** Sólo uno de los bloques ($B_1, B_2, \dots B_k$) se ejecutará.

```
if condition1:  
    B1  
elif condition2:  
    B2  
...  
else:  
    Bk
```

$$T_{\text{if-else}}(n) = \max(T_{B_1}(n), T_{B_2}(n), \dots, T_{B_k}(n))$$

Eficiencia de Algoritmos

```
def sum_n(n: int) -> int:
    """returns the sum of the first n numbers"""
    result = 0          # 1
    for j in range(n+1): # loop will be executed n+1 times
        result += j     # 2

    return result      # 1
```

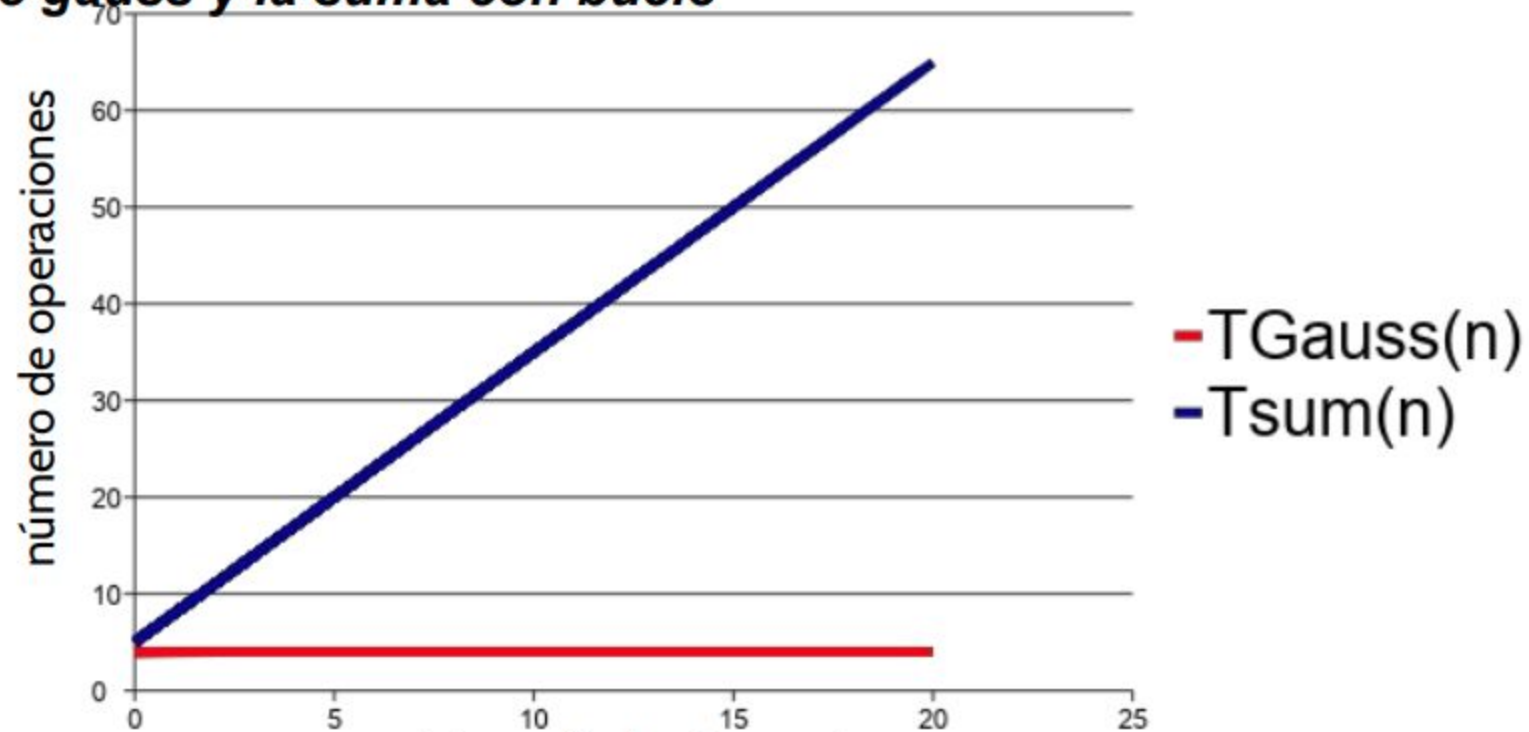
- Por tanto, $T(n) = 2(n+1) + 2 = 2n + 4$
- Depende del valor de n .

```
def sum_gauss(n: int) -> int:
    """returns the sum of the first n numbers, by
    applying Gauss sum"""
    return n*(n+1) // 2
```

- En este caso, $T(n) = 1 +$ (suma)
1 + (multiplicación)
1 + (división entera)
1 (return)
= 4
- $T(n) = 4$, no depende del valor de n .

Eficiencia de Algoritmos

Comparativa de las funciones temporales de la suma de gauss y la suma con bucle



$$T_{sum}(n) = 2n + 4$$
$$T_{Gauss}(n) = 4$$

Eficiencia de Algoritmos

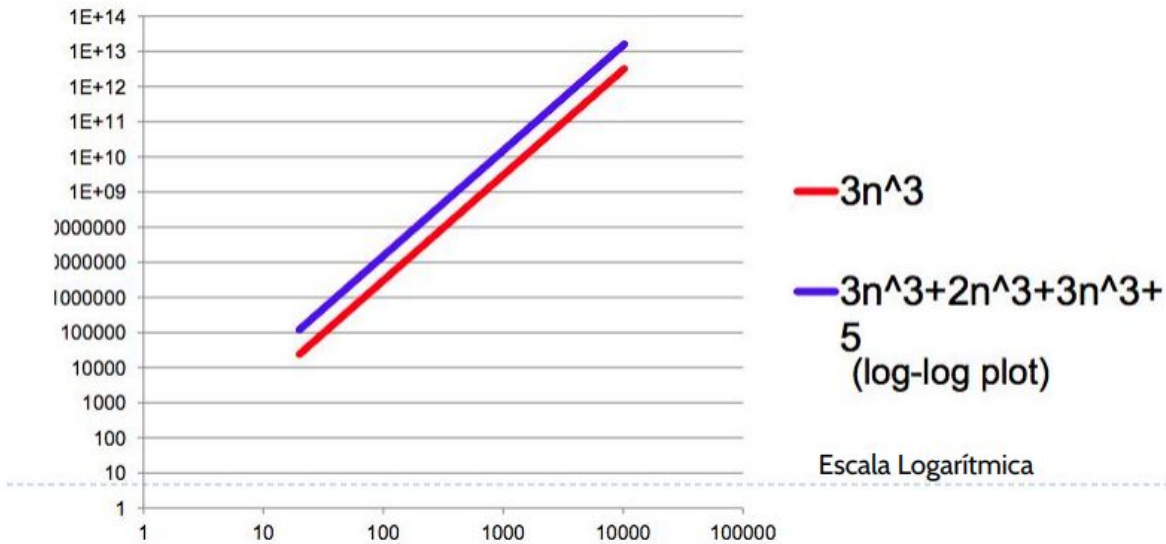
- Supón que tienes dos algoritmos, A y B, con las siguientes funciones de tiempo:
 - $T_A(n) = 3n^3$
 - $T_B(n) = 3n^3 + 2n^2 + 3n + 5$

¿Qué algoritmo es más eficiente?

Eficiencia de Algoritmos

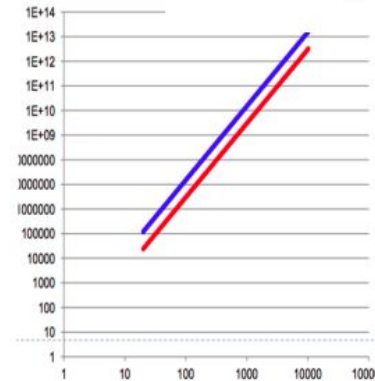
$$T_A(n) = 3n^3$$

$$T_B(n) = 3n^3 + 2n^2 + 3n + 5$$



- Dos funciones, f y g , son asintóticamente equivalentes cuando:

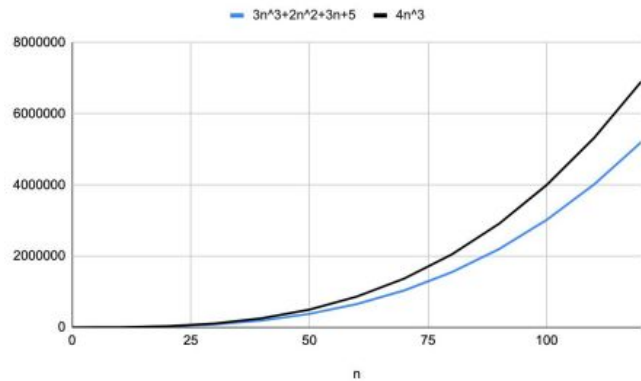
$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1$$



$$\lim_{n \rightarrow \infty} \frac{3n^3 + 2n^2 + 3n + 5}{3n^3} = 1$$

Eficiencia de Algoritmos

$T_B(n) = 3n^3 + 2n^2 + 3n + 5$ es de orden superior n^3 porque existen $n_0 = 10$, $c = 4$, tales que $T_B(n) \leq cn^3$, para todo $n \geq n_0$



n	$3n^3 + 2n^2 + 3n + 5$	$4n^3$
0	5	0
5	445	500
10	3235	4000
20	24865	32000
30	82895	108000
40	195325	256000
50	380155	500000
60	655385	864000
70	1039015	1372000
80	1549045	2048000
90	2203475	2916000
100	3020305	4000000
110	4017535	5324000
120	5213165	6912000

Buscar un límite superior a la función $T(n)$

1. Buscar el término que crece más rápido.
2. Eliminar su coeficiente.

- $T_A(n) = 3n^3 \rightarrow 3n^3 \rightarrow n^3$
- $T_B(n) = 3n^3 + 2n^2 + 3n + 5 \rightarrow 3n^3 \rightarrow n^3$

Cota superior también se llama función **Big O**

Eficiencia de Algoritmos

T(n)	BigO
4	$O(1)$
$3n+4$	$O(n)$
$5n^2+ 27n + 1005$	$O(n^2)$
$10n^3+ 2n^2 + 7n + 1$	$O(n^3)$
$n!+ n^5$	$O(n!)$

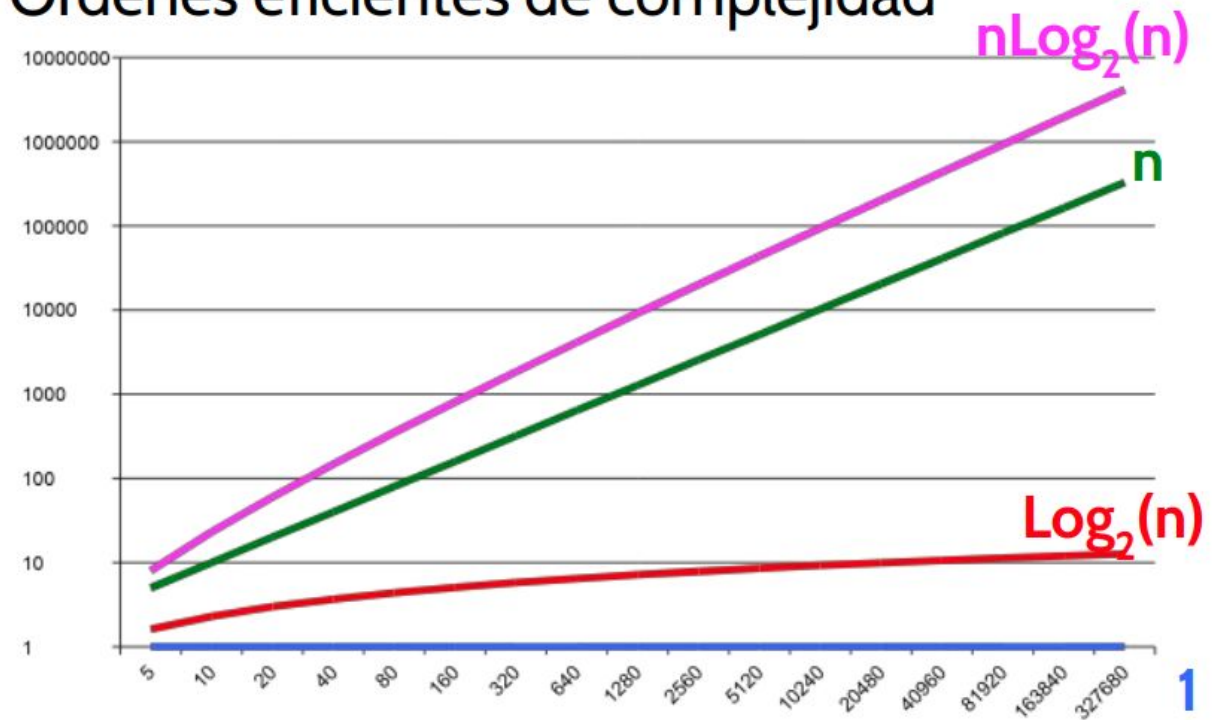
T(n)	Big-O
$n + 2$	$O(n)$
$\frac{1}{2}(n+1)(n-1)$	$O(n^2)$
$7n^4+5n^2+1$	$O(n^4)$
$n(n-1)$	$O(n^2)$
$3n+\log(n)$	$O(n)$

Eficiencia de Algoritmos

Big O funciones

notación	nombre
$O(1)$	constante
$O(\log n)$	logarítmico
$O(n)$	lineal
$O(n \log n)$	lineal-logarítmico
$O(n^2)$	cuadrática
$O(n^c)$	polinómico
$O(c^n)$	exponencial
$O(n!)$	factorial

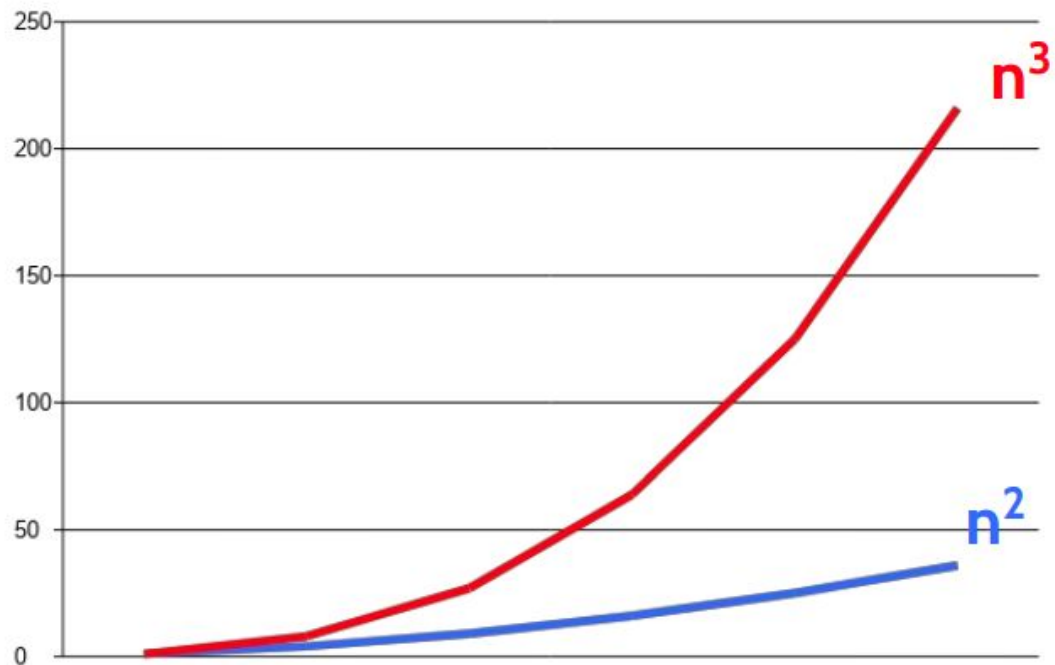
Órdenes eficientes de complejidad



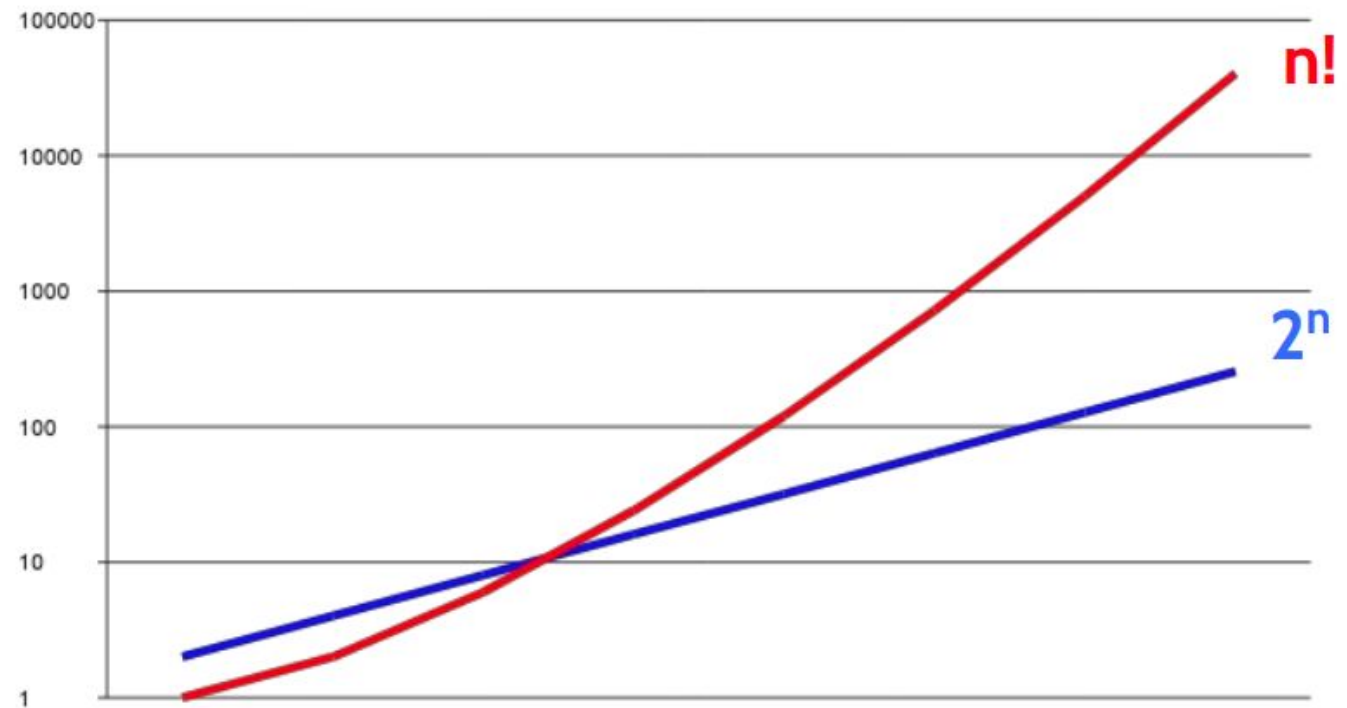
$$1 < \log n < n < n \log n < n^2 < n^3 < \dots < 2^n < n!$$

Eficiencia de Algoritmos

Órdenes tratables de complejidad



Órdenes intratables de complejidad:



Eficiencia de Algoritmos - Ejercicios

Construir algoritmos que tengan las complejidades de cada ejercicio. Luego encontrar qué algoritmos tienen complejidad equivalente y cuáles no; y cual es el más complejo.

1. Ejercicio 1:

$$f(n) = 3n^2 + 5n + 2$$

2. Ejercicio 2:

$$f(n) = 2n^3 + 7n^2 + n$$

3. Ejercicio 3:

$$f(n) = 10n \log(n) + n^2$$

4. Ejercicio 4:

$$f(n) = 5n + \sqrt{n}$$

5. Ejercicio 5:

$$f(n) = 2^{n+1} + n^2$$

6. Ejercicio 6:

$$f(n) = 50 + n^3$$